LAPORAN IMPLEMENTASI & VISUALISASI STEP-BY-STEP ALGORITMA SEARCHING DAN SORTING DATA NILAI MAHASISWA PADA C# WINDOWS FORM

Nama : Aysila Zukriya Nafi' (2511002)

Rohman Wahyu Pratama (2511013)

Mata Kuliah : Pemrograman Lanjut

Prodi : Informatika

A. Analisis Searching

1. Perbandingan Sequential vs Binary Search vs Brute Force

Uji pencarian nilai 78 (posisi awal), 68 (posisi tengah), dan 73 (posisi akhir). Buat tabel berisi jumlah langkah tiap algoritma untuk ketiga nilai tersebut.

Nilai	Target Posisi	Langkah Sequential Search	Langkah Binary Search	Langkah Brute Force
78	Awal (Indeks 0)	1 Langkah	4 Langkah	1 Langkah
68	Tengah (Indeks 20)	21 Langkah	1 Langkah	21 Langkah
73	Akhir (Indeks 24)	25 Langkah	5 Langkah	25 Langkah

Keterangan: Brute Force merupakan metodee pencarian dengan cara mencoba satu per satu, Pada program ini, brute force diimplementasikan dalam bentuk sequential search, sehingga jumlah langkahnya sama.

2. Kasus Nilai Duplikat

Cari nilai 55 dan 91 menggunakan Sequential Search. Apakah kedua data ditemukan? Jelaskan bagaimana Binary Search menangani duplikat.

Jawab:

Ya, tetapi secara visual dan prosedural, Sequential Search dalam kode akan terus berjalan hingga akhir array (indeks ke-24).

- **Nilai 55:** Ditemukan dua kali, yaitu pada **indeks 1** (Budi) dan **indeks 12** (Miko).

- **Nilai 91:** Ditemukan dua kali, yaitu pada **indeks 8** (Ilham) dan **indeks 18** (Sari).

Cara Binary Search Menangani Duplikat:

- Data diurutkan terlebih dahulu, nilai duplikat akan berdekatan dan hanya menmeukan salah satu posisi.

3. Kesimpulan Searching

Algoritma mana yang lebih efisien dan dalam kondisi apa masing-masing lebih unggul?

Jawab:

a) Perbandingan Efisiensi
Secara umum, Binary Search jauh lebih efisien daripada Sequential Search dalam hal kecepatan (jumlah langkah), terutama untuk dataset yang besar.

b) Kondisi Unggul Masing-Masing
Sequential Search (Linear Search)
Algoritma ini lebih unggul dalam kondisi:

- Data Tidak Terurut:
- Dataset Kecil
- Mencari Semua Duplikat

Binary Search
Algoritma ini lebih unggul dalam kondisi:

- Data Sudah Terurut.
- Dataset Skala Besar
- Pencarian Berulang

B. Analisis Sorting

1. Tabel Perbandingan Hasil

Isi tabel jumlah perbandingan dan pertukaran dari keempat algoritma berdasarkan hasil program.

Algoritma	**Perbandingan**	**Pertukaran**
Bubble Sort	~270	~140
Selection Sort	300	22

Insertion Sort	~170	~140
Counting Sort	0	25

2. Analisis Tiap Algoritma

Jelaskan pola langkah yang terlihat saat visualisasi dan apakah banyak pertukaran yang tidak perlu terjadi.

- **Bubble Sort:** Pola ini sangat tidak efisien karena satu nilai besar harus ditukar berkali-kali ke posisi kanan.
- **Selection Sort:** Algoritma ini meminimalkan pertukaran posisi, namun perbandingannya tetap tinggi.
- **Insertion Sort:** Memindahkan elemen ke posisi yang tepat, ini lebih efisien untuk data kecil (hamper terurut).
- **Counting Sort:** Algoritma ini tidak membandingkan antar elemen tetapi langsung menempatkan mahasiswa ke posisi yang benar berdasarkan hitungan frekuensi nilai.

3. Pengaruh Data Tidak Terurut

Data awal sengaja acak. Jelaskan bagaimana kondisi ini mempengaruhi performa tiap algoritma dan algoritma mana yang paling banyak melakukan langkah ekstra.

a) **Bubble Sort (Paling Banyak Langkah Ekstra):** Langkah ekstra pling banyak,banyak pertukaran & perbandinfan karena geser satu persatu.
b) **Insertion Sort:** Banyak pergeseran saat data acak dan kurang efisien jika data tidak terurut.
c) **Selection Sort:** Performa perbandingannya stabil namun tidak efisien. Langkah ekstranya terletak pada proses pemindaian yang tetap dilakukan maksimal.
d) **Counting Sort (Paling Efisien):** paling efisien, tidak ada perbandingan dan tidak terpengaruh data acak.

4. Kesimpulan Sorting

- Sisi Perbandingan, Paling efisien : Counting Sort (0)
- Sisi Pertukaran, Paling sedikit : Selection Sort (~22)

- Direkomendasikan: Counting Sort karena paling cepat secara keseluruhan.

C. Kesimpulan Umum

Apa yang paling sulit diimplementasikan dan bagaimana cara mengatasinya? Sertakan pembagian tugas tiap anggota kelompok.

Jawab:

Hal Paling Sulit Diimplementasikan:

a. Membangun sistem visualisasi langkah demi langkah (Step-by-Step Visualization).
b. Btn Auto Play tidak bisa jalan.

Cara Mengatasi:

a) Penggunaan Teknik Snapshot (.Clone()):, Sistem Kontrol Timer dan CurrentStep: Untuk menghindari aplikasi membeku (*freeze*)
b) Klik tombol auto play pada form designer, buka properties, pilih tab events lalu bagian click masukkan btnAutoPlay_click dan masukkan code pada Form1.cs

Pembagian Tugas Tiap Kelompok:

a) Pengembangan tampilan (UI) dan Visualisasi : AYSILA ZUKRIYA NAFI'
b) Implementasi Algoritma : AYSILA ZUKRIYA NAFI'
c) Penyusunan analisis : ROHMAN WAHYU PRATAMA
d) Integrasi sistem dan debugging : ROHMAN WAHYU PRATAMA